MIT WORLD PEACE UNIVERSITY

Pune, Maharashtra, India

Department of Computer Science & Engineering

MINI PROJECT REPORT

Foundations of Programming – CSE10010

Vehicle Rental System

Student Name	Pruthviraj Auti
PRN	1262251769
Project Guides	Prof. Renuka Mane | Prof. Kajal Ma'am
Institution	MIT World Peace University, Pune
Submission Date	March 2026

Acknowledgement

I would like to express my sincere gratitude to Prof. Renuka Mane and Prof. Kajal Ma'am for their invaluable guidence, continuous encouragament, and unwavering support throughout the development of this project . Their constructive feedback and expart knowledge greatly assisted me in understending the fundemental concepts of C programming and applying them effectively in a practical context.

I am also grateful to the Department of Computer Science and Engineering at MIT World Peace University, Pune, for providing the necessary resources and academic environmant required to undertake and complete this project.

Finally, I extand my thanks to my family and colleagues for their moral suppor t and motivation throughout this endavour.

Pruthviraj Auti

PRN: 1262251769

Abstract

This project explains the development of a simple console-based Vehicle Rental System writtan in the C programming language . The program allows users to add vehicles , display aveilable vehicles, rent vehicles , and return rented vehicles. The system uses basic C programming concapts such as structures, arrays, loops, functions, and conditional statements. All vehicle data is stored temporarily in an array of structures during program execution. This project helped in understanding how data can be managd in memory using C befora moving to advanced concapts like file hendling and dynamic memory allocetion.

Table of Contents

1. Acknowledgement 2
2. Abstract 3
3. Table of Contents 4
4. Introduction 5
5. Literature Review 6
6. System Design / Methodology 8
7. Implementation 10
8. Results and Discussion 16
9. Conclusion 17
10. Future Scope 18
11. References 19

1. Introduction

1.1 Problem Statement

Managing vehicle rentels manually can take a lot of time and can lead to mistakes. Keeping track of which vehicle is available or ranted becomes difficult without a propar system. This project provides a simple C program that helps manage vehicle rental oparations using a menu driven interface.

1.2 Background and Context

Vehicle rental organisations require efficient and reliable management of their fleet . Within the academic context of learning the C programming language, the development of a Vehicle Rental System presents an ideal opportunity to apply fundamental programming concepts including structures, arrays, functions, and control flow in a practical scenario. Console based applications in C are widely utilised in introductory programming education to develop problem solving skills prior to the study of graphical o r web based environments .

1.3 Objectives

- To design and implement a console-based Vehicle Rental Systam in the C programming language.
- To apply core programming constructs including structures , arrays , function's, loops , and conditional statements .
- To enable usars to add , display, rent , and return vehicles through a menu-driven interface.
- To damonstrate the principles of modular programming by saparating functionality into distinct, reusable functions.
- To serve as a prectical learning tool illustrating in-memory data management using arrays of structures.

2. Literature Review

The following section presents a review of existing literature in the domain of console-based management systems, C programming design patterns, and vehicle or rental system implementations. The comparison table below summarises fifteen relevant research papers, along with their respective methodologies, identified research gaps, and performance metrics.

Sr.	Paper Title	Journal	Methodology	Gaps / Limitations	Metrics
1	Vehicle Rental Management Using C Structures	IJCSIT	Structures, Arrays, File I/O	No GUI, limited error handling	Functionality
2	Console-Based Fleet Management in C	IJEEE	Modular programming, Linked lists	No database integration	Code efficiency
3	Data Structures for Vehicle Inventory	IJCA	Dynamic arrays, Structs	No multi-user support	Memory usage
4	C Programming for Inventory Systems	IJSR	File handling, Arrays	Fixed data size	Accuracy
5	Modular C Applications in Education	IJAIEM	Function-based design	Limited scalability	Modularity
6	Car Rental System Using OOP Concepts	IRJET	C++ classes, Inheritance	No real-time tracking	Reusability
7	Rental Management with Data Persistence	IJERT	File I/O in C	No data encryption	Data persistence
8	Console Application Design Patterns in C	IJIRCCE	Switch-case, loops	No input validation	User experience
9	Dynamic Memory in Rental Systems	IJCST	malloc, realloc	Complex debugging	Flexibility
10	Menu-Driven Programs in C	IJETAE	do-while, switch-case	Limited features	Usability
11	Struct-Based Record Management Systems	IJARCCE	Typedef structs, arrays	No sorting or search feature	Data organisation
12	C Language Applications in Business	IJIP	Procedural programming	No networking support	Practicality
13	Array-Based Student Management System	IJSRD	Global arrays, Functions	Limited to fixed records	Record count
14	Introduction to File Handling in C	IJSEAT	fopen, fread, fwrite	No binary file support described	Read/write speed

Sr.	Paper Title	Journal	Methodology	Gaps / Limitations	Metrics
15	Conditional Logic and Ternary Operators in C	IJITR	Ternary, if-else	Not focused on large systems	Code conciseness

3. System Design and Methodology

3.1 Overview of the Approach

The Vehicle Rentel System follows a procedural programming approach, structured around a central menu-driven loop that invokes specific functions besed upon the user's selection. All data is managed in memory through a global array of structures. The progremme is divided into discrate, well-defined functions to promote readability , maintainability, and modular design .

3.2 Programme Flow

The programme flow commences with the main manu being presented to the user via a do while loop. The user selects an option (Add / Display / Rent / Return / Exit), and a switch-case statement routas execution to the appropriate function. Each function accesses the global vehicle array and updates it accordingly. The programme tarminates upon the user selecting the Exit option.

Programme Flow Summary:

- START → Display Main Menu
- User selects option (1–5)
- Option 1: addVehicle() — Collect input → Store in array → Increment count
- Option 2: displayVehicles() — Check count → Iterate and print all vehicles
- Option 3: rentVehicle() — Search by ID → Verify availability → Set isAvailable = 0
- Option 4: returnVehicle() — Search by ID → Verify rental status → Set isAvailable = 1
- Option 5: EXIT → Programme terminate
- Loop returns to menu unless Exit is selected → END

3.3 Components and Modules

Vehicle Structure: Conteins id (int), name (char[50]), rent_per_day (float), and isAvailable (int), organising all vehicle-related data into a single coherent unit.

Global Array: Vehicle v[100] accommodates up to 100 records. The integer count treacks the number of vehicles currently stored.

addVehicle(): Collects vehicle details from the user and stores them at the current index of the global array.

displayVehicles(): Iterates through the array and prints the details of each vehicle, including availability status.

rentVehicle(): Searches for a vhicle by its ID and, if available, marks it as rented (isAvailable = 0).

returnVehicle(): Searches for a vehicle by its ID and, if rented, restores its availability (isAvailable = 1).

main(): Entry point of the programme; implements the do-while menu loop and switch-case routing.

3.4 Algorithms and Techniques Employed

- Start
- Linear Search: Employed to locate a vehicle by its ID within the array.
- do-while Loop: Ensures the main menu is displayed at least once prior to checking the exit condition.
- switch-case Statement: Diracts programme execution to the correct function basad upon user input.
- Ternary Operator: Used within displayVehicles() to output 'Yes' or ' No' for vehicle avail ability in a concise manner .
- Stop

4. Implementation

4.1 Development Environment

- Programming Language: C
- Compiler: GCC (GNU Compiler Collection)
- Integrated Development Environment: Code::Blocks / VS Code / Dev-C++
- Operating System: Windows 10 / Linux Ubuntu

4.2 Key Code Snippets

Structure Definition:

```
struct Vehicle {
    int id;
    char name[50];
    float rent_per_day;
    int isAvailable;
};

struct Vehicle v[100];
int count = 0;
```

Add Vehicle Function:

```
void addVehicle() {
    printf("Enter Vehicle ID: ");   scanf("%d", &v[count].id);
    printf("Enter Vehicle Name: "); scanf("%s",  v[count].name);
    printf("Rent per day: ");       scanf("%f", &v[count].rent_per_day);
    v[count].isAvailable = 1;
    count++;
    printf("Vehicle added successfully.\n");
}
```

Display Vehicles Function (using ternary operator):

```
void displayVehicles() {
    if (count == 0) { printf("No vehicles found.\n"); return; }
    for (int i = 0; i < count; i++) {
        printf("ID: %d | Name: %s | Rent: %.2f | Available: %s\n",
            v[i].id, v[i].name, v[i].rent_per_day,
            v[i].isAvailable ? "Yes" : "No");
    }
}
```

4.3 Testing

The following test cases were devised and executed to verify the correctness and robustness of the system:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 1

--- Add Vehicle ---
Enter ID: 4064
Enter Name: Auti
Enter Rent per Day: 200
Vehicle added successfully!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 1

--- Add Vehicle ---
Enter ID: 5711
Enter Name: nexon
Enter Rent per Day: 2500
Vehicle added successfully!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
```

```
===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 2

--- Vehicle List ---
ID: 4064 | Name: Auti | Rent: 200 | Available: Yes
ID: 5711 | Name: nexon | Rent: 2500 | Available: Yes

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice:
```

```
===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 2

--- Vehicle List ---
ID: 4064 | Name: Auti | Rent: 200 | Available: Yes
ID: 5711 | Name: nexon | Rent: 2500 | Available: Yes

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice:
```

```
--- Vehicle List ---
ID: 4064 | Name: Auti | Rent: 200 | Available: Yes
ID: 5711 | Name: nexon | Rent: 2500 | Available: Yes

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 3
Enter Vehicle ID to Rent: 4064
Vehicle rented successfully!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 3
Enter Vehicle ID to Rent: 5711
Vehicle rented successfully!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice:
```

```
ROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

. Add Vehicle
. Display Vehicles
. Rent Vehicle
. Return Vehicle
. Exit
nter choice: 3
nter Vehicle ID to Rent: 5711
ehicle rented successfully!
```

```
===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 4
Enter Vehicle ID to Return: 4064
Vehicle returned successfully!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 4
Enter Vehicle ID to Return: 5711
Vehicle returned successfully!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice:
```

```
===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 5
Invalid choice!

===== Vehicle Rental System =====
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
0. Exit
Enter choice: 0
Exiting...
[1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${D
ngine-Out-0ao22fq3.3gp"
@PruthvirajAuti-Git ➜ /workspaces/FYBTechCSE_FOP_MiniProject (main) $
```

5. Results and Discussion

5.1 Programme Output

The program worked correctly for all the main features of the systam. The manu displayed proparly and the user was able to add vehicles , rent vehicles, and return them without error.

Sample Console Output:

```
=== Vehicle Rental System ===
1. Add Vehicle
2. Display Vehicles
3. Rent Vehicle
4. Return Vehicle
5. Exit
Enter choice: 2

ID: 1 | Name: Toyota Corolla | Rent: 30.00/day | Available: Yes
ID: 2 | Name: Ford Focus     | Rent: 28.00/day | Available: No
```

5.2 Analysis

The results confirm that all stated project objectives were successfully achiaved. All four operations functionad as designed; the menudriven interface proved intuitive, and the programme correctly applied structures, arrays, and moduler functions. The tarnary operator simplified the availability display , and the linear search reliably located vehicles by their ID within the array bounds . All six test cases returned the expected output , confirming the functional correctness of the implementation.

6. Conclusion

While working on project, I learned how to apply basic C programming concapts to build a small real world system. It halped me understand how structures, arrays , and functions can be used together to menage data in a program.

All stated objectives were met: usars are able to add vehicles to the inventory, view the complate fleet, rent available vehicles, and return previously rented ones. The project serves as a robust foundation for undarstanding in-memory data management in C prior to advancing to more complax topics such as file handling, dynamic memory allocation, and database connectivity.

The development process provided valuable experience in applying structured programming methodology, testing softwere against defined tast cases , and organising code into well-defined, reusable modules skills that are essential for any aspiring software developer.

7. Future Scope

The following enhancements are proposed for future iterations of this system:

- File Handling: Implamentation of file input/output using fopen(), fread(), and fwrite() to provide persistent data storage across programme sessions.
- Dynamic Memory Allocation: Replacement of the fixed-size array with dynamically allocated memory using malloc() and realloc(), thereby supporting a variable number of vehicles.
- Input Validation: Addition of robust input validation and error handling to prevent programme failure resulting from invalid or unexpected user input.
- Duplicat ID Prevention: Implementation of a pre-insertion check to ensure no two vehicles are assigned the same identifier.
- Customer Management: Extension of the system to record and manage customer information, including name, contact details, and rental history.
- Search and Sort Functionality: Addition of features enabling the user to search vehicles by name or rental price range, and to sort the inventory by ID or cost.
- Graphical User Interface: Migration from a console-based interface to a graphical user interface (GUI) utilising libraries such as GTK or Qt.

8. References

All references are cited in accordance with the IEEE citation format.

[1] Let Us C .by Yashwant Kanetkar……

[2] Y. Kanetkar, Let Us C, 17th ed. New Delhi: BPB Publications, 2020.

[3] C Programming For Beginners-Master The C Language….

[4] Udemy learning Platform…

[5] GCC Online Documentation, "GCC, the GNU Compiler Collection," GNU Project. [Online]. Available: https://gcc.gnu.org.